

AI-Powered Banking Transaction Processing Pipeline

HW6 — Final Capstone

Roman Popyk · July 2026

Agenda

1. Overview — What is this project?
2. Architecture — How the pipeline works
3. Pipeline Stages — Validator, Fraud Detector, Reporting
4. Workflow Agents — Spec, Code Gen, Tests, Docs
5. Demo Results — Real pipeline run output
6. MCP Integration — context7 + custom server
7. Coverage Gate — Husky pre-push hook
8. Lessons Learned

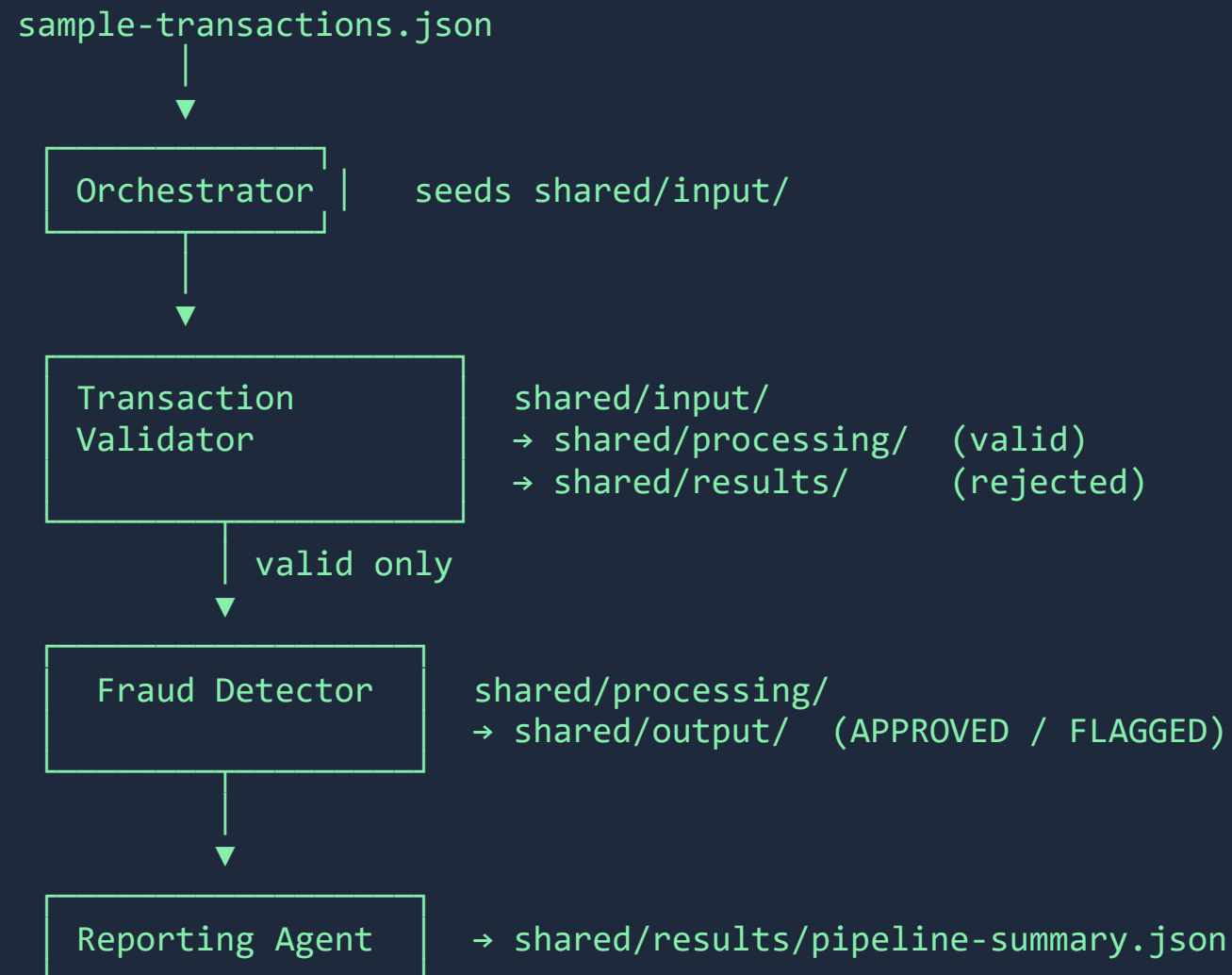
Overview

An AI-assisted, multi-stage TypeScript pipeline that processes raw banking transactions through automated validation, fraud detection, and reporting — built entirely using AI workflow agents.

Key facts

Language	TypeScript 5 / Node.js 22 LTS
Agents	4 workflow agents (Spec, Code, Tests, Docs)
Pipeline stages	3 (Validator → Fraud Detector → Reporting)
Test coverage	100% (68 unit tests)
Coverage gate	Husky pre-push blocks pushes below 80%
MCP servers	2 (context7 + custom pipeline-status)

Architecture



File-Based Message Protocol

Each stage communicates via **JSON** files in shared directories. No direct function calls between stages.

```
{
  "message_id": "e800d3fc-de6e-4fb1-bd85-0763c7c653b5",
  "timestamp": "2026-03-16T10:00:00Z",
  "source_stage": "validator",
  "target_stage": "fraud_detector",
  "message_type": "transaction",
  "data": {
    "transaction_id": "TXN001",
    "amount": "1500.00",
    "currency": "USD",
    "status": "validated"
  }
}
```

Stages are **fully decoupled** — any stage can be replaced independently.

Stage 1 — Transaction Validator

Validates every raw transaction before it enters the pipeline.

Checks performed

Check	Rule
Required fields	<code>transaction_id</code> , <code>timestamp</code> , <code>source_account</code> , <code>destination_account</code> , <code>amount</code> , <code>currency</code> , <code>transaction_type</code>
Amount	Must be a positive <code>decimal.js</code> value (no float!)
Currency	Must be a valid ISO 4217 code (150+ codes supported)
Timestamp	Must be ISO 8601 format

Outcomes

- ✓ Valid → written to `shared/processing/` for fraud detection

Stage 2 — Fraud Detector

Scores each validated transaction across four risk factors.

Risk scoring model

Factor	Score	Trigger
High value	+40	Amount > \$10,000
Cross-border	+30	Country ≠ US
Off-hours	+20	UTC hour < 6 or > 22
Wire transfer	+10	transaction_type = wire_transfer

Decision threshold

Score	Status
> 60	

Stage 3 — Reporting Agent

Aggregates all outcomes into a consolidated pipeline summary.

What it reads

- `shared/output/*.json` — approved and flagged transactions
- `shared/results/*-rejected.json` — rejected transactions

Output: `pipeline-summary.json`

```
{
  "generated_at": "2026-07-07T15:57:53Z",
  "total": 8,
  "approved": 6,
  "flagged_for_review": 0,
  "rejected": 2,
  "by_currency": {
    "USD": { "count": 5, "total_amount": "114699.99" },
    "EUR": { "count": 1, "total amount": "500.00" }
```


Four Workflow Agents

Agent	Role	Deliverable
Agent 1 — Spec	Writes technical specification	<code>specification.md</code> + <code>/write-spec</code> slash command
Agent 2 — Code	Generates pipeline code	Orchestrator, 3 stages, frontend, research notes
Agent 3 — Tests	Writes unit tests	68 tests, 100% coverage, Husky coverage gate
Agent 4 — Docs	Generates documentation	<code>README.md</code> , <code>HOWTORUN.md</code> , this presentation

Each agent is backed by a **skill**, **MCP server**, **hook**, or **requirement** that was fulfilled during the build.

Demo — Pipeline Run

Running `npm run pipeline` on 8 sample transactions:

AI-Powered Banking Transaction Pipeline

- ▶ Loading 8 transactions from `sample-transactions.json`
- ▶ Stage 1: Transaction Validator
- ▶ Stage 2: Fraud Detector
- ▶ Stage 3: Reporting Agent

==== Pipeline Summary =====

Total transactions:	8
Approved:	6
Flagged for review:	0
Rejected:	2

By Currency:

USD:	5 transactions,	total \$114,699.99
EUR:	1 transaction,	total €500.00

Demo — Rejection Examples

TXN006 — Invalid currency

```
{  
  "transaction_id": "TXN006",  
  "status": "REJECTED",  
  "reason": "Unknown ISO 4217 currency code: XYZ"  
}
```

TXN007 — Negative amount

```
{  
  "transaction_id": "TXN007",  
  "status": "REJECTED",  
  "reason": "Amount must be a positive number, got: -100.00"  
}
```

All rejected transactions land in `shared/results/` with a machine-readable `reason`.

MCP Integration

Two MCP servers configured in `mcp.json`:

1. context7 (development aid)

Used during code generation to look up:

- `decimal.js` — monetary arithmetic patterns (`ROUND_HALF_UP` , string construction)
- Node.js `fs/promises` — async file I/O for the file-based pipeline protocol

2. pipeline-status (custom server — `mcp/server.ts`)

Endpoint	Type	Description
<code>get_transaction_status</code>	Tool	Returns status + details for a transaction ID
<code>list_pipeline_results</code>	Tool	Returns full pipeline summary
<code>pipeline://summary</code>	Resource	Latest summary JSON as text

Custom MCP Server — Live Query

Querying `get_transaction_status` for TXN001 via JSON-RPC:

```
// Request
{"method": "tools/call", "params": {
  "name": "get_transaction_status",
  "arguments": {"transaction_id": "TXN001"}
}}

// Response
{
  "transaction_id": "TXN001",
  "status": "APPROVED",
  "location": "shared/output",
  "details": {
    "amount": "1500.00",
    "currency": "USD",
    "risk_score": 0
  }
}
```

Coverage Gate — Husky Pre-Push Hook

`.husky/pre-push` fires on every `git push`:

```
#!/bin/sh
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
PROJECT_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
cd "$PROJECT_DIR" || exit 1

echo "Running test coverage check before push..."
npm run test:coverage

if [ $? -ne 0 ]; then
    echo "Push blocked: coverage below 80%"
    exit 1
fi
echo "Coverage check passed."
```

Coverage thresholds (in `jest.config.js`): 80% on branches, functions, lines, statements.
Current coverage: 100% across all metrics.

Test Coverage

68 unit tests across 7 test files:

Test file	Tests	What it covers
<code>test_transaction_validator.ts</code>	16	<code>validateTransaction()</code> — all rejection cases
<code>test_validator_runner.ts</code>	6	<code>runValidator()</code> — file I/O with mocked dirs
<code>test_fraud_detector.ts</code>	16	<code>computeRiskScore()</code> , <code>detectFraud()</code>
<code>test_fraud_runner.ts</code>	7	<code>runFraudDetector()</code> — file I/O
<code>test_reporting_agent.ts</code>	8	<code>generateReport()</code> — aggregation logic
<code>test_logger.ts</code>	5	<code>maskAccount()</code> , <code>auditLog()</code>
<code>test_fileio.ts</code>	10	All filesystem utilities

Result: 100% coverage — statements, branches, functions, lines

Lessons Learned

1. decimal.js is non-negotiable for money

Native `number` loses precision with floats. Always construct `Decimal` from strings, never from literals.

2. File-based protocols make stages truly independent

Swapping a stage requires zero changes to other stages — just agree on the JSON schema.

3. Husky + Jest thresholds = automatic quality gate

Coverage can never silently drop; every push is verified. Saved effort of manual checks.

4. MCP servers extend AI agent capabilities

The custom `pipeline-status` server lets AI tools query live pipeline state without

Thank You

Roman Popyk — HW6 Final Capstone

Repository

`homework-6-submission` branch

Run it

```
npm install && npm run pipeline
npm run frontend      # → http://localhost:3000
npm run mcp           # → pipeline-status MCP server
npm test              # → 68 tests, 100% coverage
```